

Project Specification: Dockerized AI Video Stitching & Clip Ordering Validation Pipeline

1. Project Background

We are building a technical validation module for an AI video production pipeline.

The business scenario is:

A used-car seller uploads a raw car walkaround video, for example a video filmed by walking around a car. The system may also generate one or more AI-narrated clips, such as an intro, a selling-point explanation, or a call-to-action segment. The final product should automatically combine the user-shot footage and AI-generated clips into a natural, coherent sales video.

The first technical goal is not to build the full AI video product. The first goal is to prove that we can build a reliable backend video-stitching module that can run repeatedly, automatically, and reproducibly in a Docker environment.

The validation should focus on two core tasks:

1. Concatenate two or more clips that were cut from a complete video and are already provided in the correct chronological order.
2. Stitch two or more clips that were cut from a complete video but are provided in a jumbled order, by automatically recovering the correct sequence before stitching them.

These two tasks represent the basic technical foundation for later assembling user-uploaded car footage and AI-generated narrated clips.

2. High-Level Objective

Build a Dockerized video processing worker that can:

1. Accept multiple video clips as input.
2. Normalize the clips into a consistent format.
3. Concatenate clips in a known correct order.
4. Infer the correct order of shuffled clips cut from the same original video.
5. Stitch the ordered clips into a final MP4 video.
6. Produce structured logs and metadata for debugging and evaluation.
7. Be runnable in a clean Docker environment without relying on local machine configuration.

The key product-level value is:

Not just "can we manually concatenate videos," but:

Can we build a reusable backend video-stitching module that can be called by the AI video pipeline repeatedly, reliably, and automatically?

3. Technical Stack

Use the following stack unless there is a strong reason to choose otherwise:

- Language: Python 3.11+
- Video engine: FFmpeg and FFprobe
- Computer vision: OpenCV
- Optional numeric processing: NumPy
- Optional API framework: FastAPI, only if useful
- Packaging: Docker
- Output format: MP4, H.264 video, AAC audio
- Configuration format: JSON

The MVP should work as a command-line worker first. A REST API can be added later, but the core logic should not depend on a web server.

4. Required Docker Environment

Create a Docker image that includes:

- Python 3.11+
- FFmpeg
- FFprobe
- OpenCV-compatible dependencies
- Required Python packages from `requirements.txt`

The project must be runnable through Docker using mounted input and output folders.

Example target command:

```
docker build -t ai-video-stitcher .

docker run --rm
-v "$(pwd)/input:/app/input"
-v "$(pwd)/output:/app/output"
ai-video-stitcher
python -m app.main --config /app/input/job.json
```

The Docker environment is important because the final system needs to behave like a backend worker, not like a one-off local video-editing script.

5. Functional Requirements

5.1 Mode A: Ordered Clip Concatenation

Input:

A list of video clips with known order.

Example:

```
{
  "job_id": "ordered_test_001",
  "mode": "ordered_concat",
  "clips": [
    {
      "clip_id": "clip_001",
      "path": "/app/input/clip_001.mp4",
      "order": 1
    },
    {
      "clip_id": "clip_002",
      "path": "/app/input/clip_002.mp4",
      "order": 2
    },
    {
      "clip_id": "clip_003",
      "path": "/app/input/clip_003.mp4",
      "order": 3
    }
  ],
  "output": {
    "video_path": "/app/output/final_ordered.mp4",
    "metadata_path": "/app/output/final_ordered_metadata.json"
  },
  "settings": {
    "resolution": "1280x720",
    "fps": 30,
    "video_codec": "libx264",
    "audio_codec": "aac",
    "transition": "cut"
  }
}
```

Expected behavior:

1. Read clips according to `order`.
2. Validate each file exists.
3. Normalize all clips to the same resolution, FPS, pixel format, video codec, and audio format.
4. Concatenate the normalized clips.
5. Export final MP4.
6. Write metadata JSON with clip order, durations, processing time, and FFmpeg logs.

This mode proves the basic technical stitching pipeline.

5.2 Mode B: Shuffled Clip Reordering and Stitching

Input:

A list of video clips cut from the same original video, but provided in random order.

Example:

```
{
  "job_id": "shuffle_test_001",
  "mode": "shuffled_reorder_concat",
  "clips": [
    {
      "clip_id": "clip_c",
      "path": "/app/input/clip_c.mp4"
    },
    {
      "clip_id": "clip_a",
      "path": "/app/input/clip_a.mp4"
    },
    {
      "clip_id": "clip_d",
      "path": "/app/input/clip_d.mp4"
    },
    {
      "clip_id": "clip_b",
      "path": "/app/input/clip_b.mp4"
    }
  ],
  "output": {
    "video_path": "/app/output/final_reordered.mp4",
    "metadata_path": "/app/output/final_reordered_metadata.json"
  },
  "settings": {
```

```
    "resolution": "1280x720",
    "fps": 30,
    "video_codec": "libx264",
    "audio_codec": "aac",
    "transition": "cut",
    "ordering_method": "visual_boundary_similarity"
  }
}
```

Expected behavior:

1. Extract the first and last frame of each clip.
2. Compare the last frame of each clip against the first frame of every other clip.
3. Estimate which clip should follow which clip.
4. Recover the most likely chronological sequence.
5. Normalize clips.
6. Concatenate them in the predicted order.
7. Export final MP4.
8. Write metadata JSON containing:
 9. predicted order
 10. pairwise transition scores
 11. confidence score
 12. processing time
 13. warnings or failure reasons

This mode validates the beginning of semantic sequencing.

6. Clip Ordering Algorithm for MVP

Implement a practical MVP algorithm based on visual boundary similarity.

6.1 Frame Extraction

For each clip:

- Extract the first frame.
- Extract the last frame.
- Optionally extract 3–5 frames near the beginning and end for more stable comparison.

Use FFmpeg or OpenCV.

Example FFmpeg commands:

```
ffmpeg -y -i clip.mp4 -frames:v 1 first.png  
ffmpeg -y -sseof -0.1 -i clip.mp4 -update 1 last.png
```

6.2 Visual Similarity Score

For each possible pair `(clip_i, clip_j)`:

Compare:

```
last frame of clip_i  
vs.  
first frame of clip_j
```

Use one or more of the following methods:

1. Color histogram similarity
2. Perceptual hash similarity
3. Structural similarity approximation
4. ORB/SIFT-style feature matching if feasible
5. Optical-flow-like direction estimation if useful later

For MVP, implement at least histogram similarity and perceptual hash similarity.

A simple score can be:

```
transition_score(i, j) =  
    0.6 * histogram_similarity(i.last, j.first)  
+ 0.4 * perceptual_hash_similarity(i.last, j.first)
```

Make sure `i != j`.

6.3 Sequence Recovery

Given N clips, build a directed transition score matrix.

The goal is to find the most likely path that uses each clip exactly once.

For MVP:

- If clip count ≤ 8 , use brute-force permutation search and choose the order with the highest total transition score.
- If clip count > 8 , use greedy ordering:

- Find likely starting clip.
- Repeatedly append the unused clip with the best transition score.
- Return the predicted order.

For validation tasks, clip count will usually be small, so brute-force is acceptable.

6.4 Start Clip Detection

If no metadata is available, start clip detection is hard.

For MVP, use the following heuristic:

For each clip, calculate:

```
incoming_score = max transition score from any other clip into this clip
outgoing_score = max transition score from this clip into any other clip
```

The likely first clip may have relatively low incoming score and reasonable outgoing score.

However, if brute-force permutation is used, simply test all possible clip orders and choose the best total path score.

6.5 Ground Truth Evaluation

For test clips cut from a known full video, optionally allow a ground truth order in the config:

```
"ground_truth_order": ["clip_a", "clip_b", "clip_c", "clip_d"]
```

Then output:

```
"evaluation": {
  "exact_order_match": true,
  "position_accuracy": 1.0,
  "adjacency_accuracy": 1.0
}
```

This makes the validation objective and measurable.

7. Video Normalization Requirements

Before concatenation, every clip must be normalized to avoid FFmpeg concat errors.

Normalize each clip to:

- Resolution: default 1280x720
- FPS: default 30
- Pixel format: yuv420p
- Video codec: H.264
- Audio codec: AAC
- Audio sample rate: 48000 Hz
- Audio channels: stereo

If a clip has no audio, add silent audio so all clips have consistent audio streams.

Example normalization command:

```
ffmpeg -y -i input.mp4
-vf "scale=1280:720:force_original_aspect_ratio=decrease,pad=1280:720:(ow-iw)/
2:(oh-ih)/2,fps=30,format=yuv420p"
-c:v libx264 -preset fast -crf 23
-c:a aac -ar 48000 -ac 2
output_normalized.mp4
```

If input has no audio, generate silent audio:

```
ffmpeg -y -i input.mp4
-f lavfi -i anullsrc=channel_layout=stereo:sample_rate=48000
-shortest
-vf "scale=1280:720:force_original_aspect_ratio=decrease,pad=1280:720:(ow-iw)/
2:(oh-ih)/2,fps=30,format=yuv420p"
-c:v libx264 -preset fast -crf 23
-c:a aac -ar 48000 -ac 2
output_normalized.mp4
```

8. Concatenation Requirements

Use FFmpeg concat demuxer after normalization.

Generate a `concat.txt` file:

```
file '/app/tmp/clip_001_normalized.mp4'
file '/app/tmp/clip_002_normalized.mp4'
file '/app/tmp/clip_003_normalized.mp4'
```


Then run:

```
ffmpeg -y -f concat -safe 0 -i concat.txt -c copy /app/output/final.mp4
```

For MVP, use hard cuts only.

Transitions such as fade, crossfade, and motion blur can be added later, but they are not required for the first validation.

9. Optional Mode C: Timeline Assembly for AI Video Pipeline

After Mode A and Mode B are working, add a simple timeline assembly mode.

This mode simulates the real car video use case:

```
{
  "job_id": "car_sales_video_001",
  "mode": "timeline_assembly",
  "clips": [
    {
      "clip_id": "ai_intro",
      "path": "/app/input/ai_intro.mp4",
      "role": "ai_generated_intro",
      "order": 1
    },
    {
      "clip_id": "user_walkaround",
      "path": "/app/input/user_walkaround.mp4",
      "role": "user_uploaded_car_footage",
      "order": 2
    },
    {
      "clip_id": "ai_selling_points",
      "path": "/app/input/ai_selling_points.mp4",
      "role": "ai_generated_narration",
      "order": 3
    }
  ],
  "output": {
    "video_path": "/app/output/final_car_sales_video.mp4",
    "metadata_path": "/app/output/final_car_sales_video_metadata.json"
  },
  "settings": {
    "resolution": "1280x720",
```

```
"fps": 30,  
"transition": "cut"  
}  
}
```

This mode does not need to automatically decide where the AI clips go yet. It only proves that the pipeline can assemble user clips and AI-generated clips according to a structured timeline.

10. Project Structure

Create a clean project structure:

```
ai-video-stitcher/  
  Dockerfile  
  requirements.txt  
  README.md  
  app/  
    __init__.py  
    main.py  
    config.py  
    ffmpeg_utils.py  
    video_probe.py  
    normalizer.py  
    stitcher.py  
    frame_extractor.py  
    similarity.py  
    order_solver.py  
    evaluator.py  
    metadata.py  
    errors.py  
  tests/  
    test_config.py  
    test_order_solver.py  
    test_similarity.py  
  examples/  
    ordered_job.json  
    shuffled_job.json  
    timeline_job.json  
  scripts/  
    make_test_clips.py
```

11. Main CLI Interface

The core CLI should be:

```
python -m app.main --config /app/input/job.json
```

Optional flags:

```
python -m app.main
  --config /app/input/job.json
  --workdir /app/tmp
  --verbose
```

The CLI should:

1. Load config.
2. Validate input.
3. Create working directory.
4. Determine mode.
5. Process clips.
6. Export final video.
7. Write metadata JSON.
8. Exit with code 0 if successful.
9. Exit with non-zero code if failed.

12. Metadata Output

Every job must produce a metadata JSON file.

Example:

```
{
  "job_id": "shuffle_test_001",
  "mode": "shuffled_reorder_concat",
  "status": "success",
  "input_clip_count": 4,
  "predicted_order": ["clip_a", "clip_b", "clip_c", "clip_d"],
  "ground_truth_order": ["clip_a", "clip_b", "clip_c", "clip_d"],
  "evaluation": {
    "exact_order_match": true,
    "position_accuracy": 1.0,
    "adjacency_accuracy": 1.0
  }
}
```

```
},
"clip_metadata": [
  {
    "clip_id": "clip_a",
    "duration_seconds": 5.02,
    "width": 1920,
    "height": 1080,
    "fps": 30
  }
],
"transition_scores": [
  {
    "from": "clip_a",
    "to": "clip_b",
    "score": 0.91
  }
],
"output_video_path": "/app/output/final_reordered.mp4",
"processing_time_seconds": 12.4,
"warnings": []
}
```

13. Test Clip Generation Script

Add a helper script:

```
python scripts/make_test_clips.py
--source /app/input/source.mp4
--output-dir /app/input/generated_clips
--clip-length 5
--shuffle true
```

The script should:

1. Take a complete source video.
2. Cut it into fixed-length clips.
3. Save clips with known IDs.
4. Optionally shuffle their filenames or produce a shuffled job config.
5. Produce ground truth order in the job JSON.

This is important because the shuffled clip task needs objective evaluation.

14. Acceptance Criteria

14.1 Ordered Concatenation Acceptance Criteria

The task is successful if:

1. Docker image builds successfully.
2. The ordered job runs successfully inside Docker.
3. The final MP4 is generated.
4. The final MP4 plays normally.
5. Clip order matches the input order.
6. Metadata JSON is generated.
7. The pipeline works on at least 3 clips.
8. The pipeline works even if input clips have different resolutions or codecs.
9. The pipeline handles clips without audio by adding silent audio.

14.2 Shuffled Reordering Acceptance Criteria

The task is successful if:

1. Docker image builds successfully.
2. The shuffled job runs successfully inside Docker.
3. The system predicts an order for the shuffled clips.
4. The final reordered MP4 is generated.
5. Metadata JSON includes predicted order and transition scores.
6. If ground truth is provided, metadata includes evaluation metrics.
7. For simple clips cut from the same continuous video, the predicted order should be mostly correct.
8. The system should be transparent about uncertainty by outputting confidence scores and warnings.

14.3 Minimum Quality Bar

For the first version:

- Ordered concatenation should be nearly 100% reliable.
- Shuffled ordering does not need to be perfect for all videos, but it must be testable, explainable, and measurable.
- Failure cases should be logged clearly.

15. Non-Goals for MVP

Do not overbuild the first version.

The MVP does not need:

1. A full web UI.
2. Complex video transitions.

3. AI-generated narration.
4. Automatic script writing.
5. Automatic car feature recognition.
6. Perfect semantic video understanding.
7. GPU dependency.
8. Cloud storage integration.
9. User authentication.
10. Production queue system.

The core goal is a Dockerized, repeatable, measurable video-stitching and clip-ordering worker.

16. Future Extensions

After MVP, the module can be extended to support:

1. AI-generated intro/outro clips.
 2. Voiceover overlay instead of separate AI clips.
 3. Background music.
 4. Subtitle burning.
 5. Fade/crossfade transitions.
 6. Visual continuity scoring between user footage and AI clips.
 7. Scene classification, such as front view, side view, interior, dashboard, trunk, engine.
 8. Automatic selection of best user-shot segments from a long uploaded car video.
 9. Integration with ByteDance Seedance or other AI video generation APIs.
 10. A backend job queue, such as Redis Queue, Celery, BullMQ, or similar.
 11. Uploading the final video to object storage.
 12. API endpoint for creating video assembly jobs.
-

17. Important Implementation Notes

17.1 Use FFprobe Before Processing

Before normalization, inspect each clip with FFprobe and record:

- duration
- width
- height
- video codec
- audio codec
- FPS
- whether audio exists

17.2 Avoid `-c copy` Before Normalization

Direct concat using `-c copy` may fail if clips have different codecs, FPS, timestamps, pixel formats, or audio streams. Always normalize first.

17.3 Keep Intermediate Files

During development, keep normalized clips, extracted frames, and concat files in a working directory. This makes debugging easier.

17.4 Add Clear Error Messages

If a clip is missing, corrupted, unsupported, or cannot be decoded, return a clear error in metadata.

17.5 Make the System Deterministic

Given the same input clips and config, the output order and metadata should be reproducible.

18. Suggested Development Phases

Phase 1: Docker and Basic FFmpeg Worker

Deliverables:

- Dockerfile
- requirements.txt
- CLI entrypoint
- FFmpeg availability check
- FFprobe utility
- basic metadata output

Phase 2: Ordered Concatenation

Deliverables:

- Config parser
- input validation
- clip normalization
- concat file generation
- final MP4 export
- metadata JSON

Phase 3: Test Clip Generator

Deliverables:

- script to cut a complete video into clips
- ordered job JSON generation
- shuffled job JSON generation
- ground truth order output

Phase 4: Shuffled Clip Ordering

Deliverables:

- first/last frame extraction
- histogram similarity
- perceptual hash similarity
- transition score matrix
- brute-force order solver for small N
- greedy fallback for larger N
- evaluation against ground truth

Phase 5: Timeline Assembly Demo

Deliverables:

- simple timeline assembly config
- ability to combine user footage and AI-generated clips in explicit order
- final demo video output

19. README Requirements

Write a clear README that explains:

1. What the project does.
2. Why Docker is used.
3. How to build the Docker image.
4. How to run ordered concatenation.
5. How to run shuffled reordering.
6. How to generate test clips from a source video.
7. How to interpret metadata output.
8. Current limitations.
9. Next steps.

Example README commands:


```
docker build -t ai-video-stitcher .

docker run --rm
-v "$(pwd)/input:/app/input"
-v "$(pwd)/output:/app/output"
ai-video-stitcher
python -m app.main --config /app/input/ordered_job.json

docker run --rm
-v "$(pwd)/input:/app/input"
-v "$(pwd)/output:/app/output"
ai-video-stitcher
python -m app.main --config /app/input/shuffled_job.json
```

20. Final Expected Outcome

At the end of this project, we should have a working prototype proving:

1. We can run video stitching in a Docker environment.
2. We can concatenate clips in a known order.
3. We can attempt to recover the correct order of shuffled clips cut from the same source video.
4. We can export a final MP4 automatically.
5. We can generate structured metadata for validation.
6. We have a foundation for later combining user-uploaded car walkaround footage with AI-generated narrated clips.

This project should be treated as the first reusable backend module for a larger AI video production pipeline.